PYTHON PROGRAMMING

ASSIGNMENT 2 – CO2

Control Flow, String Operations and Functions

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences, Chennai

Student Details	Information
Student Name	Nithin Chakravarthi PS
Register Number	1912260031
Course Code	CSA0805 SLOT-A
Assignment	Python Programming – Assignment 2 (CO2)
Submission Date	30.10.2026
GitHub Repository Link	https://github.com/chakravarthi2008/CSA0805A_1912260031_NITHIN-CHAKRAVARTHI-PS
Faculty Name	Hari Priya Anandakumar

Individual Assignment | CO2 | Total Marks: 50

Contents

- Q1 – Decisions with Many Boundaries
- Q2 – Loops that Count Attempts
- Q3 – Vehicle Registration Number Validator
- Q4 – Building Long Text: + or join()?
- Q5 – Functions, Parameters and Scope
- Q6 – Recursion and Its Limits
- Q7 – Diagnose, Redesign and Validate
- Submission Checklist

Report note: Replace all measurement placeholders with your own execution results. Add output screenshots and the working GitHub link before submission.

Q1 – Decisions with Many Boundaries (6 Marks)

Bloom's Level: K4 – Analyse

1.1 Predictions

The subsidy decision depends on consumption, ration-card status and whether a household member is above 80. The decision structure should return one final outcome.

Units	Ration card	Member above 80	Predicted decision
100	No	No	Full subsidy
101	Yes	No	Partial subsidy
200	Yes	No	Partial subsidy
201	No	No	No subsidy
201	No	Yes	Special subsidy
150	No	No	No subsidy
80	Yes	Yes	Full subsidy
200	No	Yes	No subsidy

1.2 Separate if vs if–elif–else

Separate if statements are evaluated independently, so overlapping conditions can produce multiple results. An if–elif–else chain stops at the first matching branch and returns one decision. Combined Boolean conditions make each rule readable; nested if statements are useful when a secondary check only applies within one slab.

1.3 Code

```
def subsidy_separate(units, ration_card, member_over_80):
    results = []
    if units <= 100:
        results.append("Full subsidy")
    if 101 <= units <= 200 and ration_card:
        results.append("Partial subsidy")
    if units > 200 and member_over_80:
        results.append("Special subsidy")
    if not results:
        results.append("No subsidy")
    return results

def subsidy_chain(units, ration_card, member_over_80):
    if units <= 100:
        return "Full subsidy"
    elif 101 <= units <= 200:
        return "Partial subsidy" if ration_card else "No subsidy"
    elif units > 200 and member_over_80:
        return "Special subsidy"
    return "No subsidy"

tests = [(100,False,False),(101,True,False),(200,True,False),
         (201,False,False),(201,False,True),(150,False,False),
         (80,True,True),(200,False,True)]
for case in tests:
    print(case, "=>", subsidy_chain(*case))
```

1.4 Alternative and Complexity

```
def subsidy_match(units, ration_card, member_over_80):
    match True:
        case _ if units <= 100:
            return "Full subsidy"
        case _ if 101 <= units <= 200 and ration_card:
            return "Partial subsidy"
```

```
        case _ if units > 200 and member_over_80:
            return "Special subsidy"
        case _:
            return "No subsidy"
```

For k slabs, an if–elif chain or sequential lookup has O(k) worst-case time and O(1) auxiliary space. A match–case statement is readable for fixed categories; guarded cases are needed for numeric ranges. Add a screenshot showing observed results for 100, 101, 200 and 201 units.

Q2 – Loops that Count Attempts (4 Marks)

Bloom's Level: K3 – Apply

A blank OTP entry must not count as an attempt. A correct OTP exits the loop; after three counted wrong attempts, the account is locked.

2.1 while Loop

```
def verify_while(correct_otp):
    attempts = 0
    while attempts < 3:
        entered = input("Enter OTP: ").strip()
        if entered == "":
            print("Blank entry ignored")
            continue
        attempts += 1
        if entered == correct_otp:
            print("OTP verified")
            break
        print("Incorrect OTP")
    else:
        print("Account locked for 30 minutes")

def audit_hook():
    pass  # Placeholder for a future optional audit action
```

2.2 for Loop

```
def verify_for(correct_otp):
    attempts = 0
    for _ in range(3):
        entered = input("Enter OTP: ").strip()
        if entered == "":
            print("Blank entry ignored")
            continue
        attempts += 1
        if entered == correct_otp:
            print("OTP verified")
            break
        print("Incorrect OTP")
        if attempts >= 3:
            break
    else:
        print("Account locked for 30 minutes")
    # For blank entries not to consume attempts, use a while loop instead.
```

The while loop is more natural because only nonblank entries increment attempts. In a for loop, range(3) advances even after continue, so blank entries consume iterations unless the logic is redesigned. break exits early, continue skips the rest of an iteration, pass does nothing, and loop–else runs when the loop finishes without break.

2.3 Extra Attempts and Recursive Alternative

```
def verify_with_security(correct_otp, security_answer):
    attempts = 0
    limit = 3
    while attempts < limit:
        entered = input("Enter OTP: ").strip()
        if not entered:
            print("Blank entry ignored")
            continue
        attempts += 1
        if entered == correct_otp:
            print("OTP verified")
            return
        print("Incorrect OTP")
        if attempts == 3:
            answer = input("Security answer: ").strip()
            if answer.lower() == security_answer.lower():
                limit += 2
    print("Account locked for 30 minutes")
```

A recursive attempt function takes O(a) time and O(a) stack space for a attempts. A loop takes O(a) time and O(1) auxiliary space. Include test evidence for blank, wrong, correct and lockout cases.

Q3 – Vehicle Registration Number Validator (6 Marks)

Bloom's Level: K4 – Analyse

Normalise the typed registration by removing spaces and hyphens and converting to uppercase. A valid form has two state letters, two district digits, one or two series letters, and four final digits.

3.1 Completed Code

```
def normalise(text):
    return text.strip().replace(" ", "").replace("-", "").upper()

def is_valid(reg):
    if len(reg) not in (9, 10):
        return False
    if not reg[:2].isalpha() or not reg[2:4].isdigit():
        return False
    series_len = len(reg) - 8
    if series_len not in (1, 2):
        return False
    if not reg[4:4 + series_len].isalpha():
        return False
    return reg[-4:].isdigit()

def format_registration(reg):
    if not is_valid(reg):
        return "INVALID"
    series_len = len(reg) - 8
    return f"{reg[:2]} {reg[2:4]} {reg[4:4+series_len]} {reg[-4:]}"

samples = ["tn 09 ab 1234", "KA-01-M-0007", "TN9AB1234",
           "TN 22 1234 AB", "mh 12 xy 9876", "A1-02-B-1234"]
```

```
for raw in samples:
    reg = normalise(raw)
    print(f"{raw:18} -> {reg:11} {format_registration(reg)}")
```

Input	Normalised	Prediction
tn 09 ab 1234	TN09AB1234	Valid: TN 09 AB 1234
KA-01-M-0007	KA01M0007	Valid: KA 01 M 0007
TN9AB1234	TN9AB1234	Invalid: district has one digit
TN 22 1234 AB	TN221234AB	Invalid: fields misplaced
mh 12 xy 9876	MH12XY9876	Valid: MH 12 XY 9876
A1-02-B-1234	A102B1234	Invalid: state code not two letters

3.2 String Concepts and Complexity

Strings are immutable: characters cannot be changed in place. text.upper() returns a new string and does not modify text unless assigned. For a valid 9-character registration, s[-4:] returns the last four digits and s[4:-4] returns the one-letter series. For a valid 10-character registration, s[-4:] returns the last four digits and s[4:-4] returns the two-letter series. Normalising and validating take O(L) time for string length L.

A regex alternative is re.fullmatch(r'[A-Z]{2}[0-9]{2}[A-Z]{1,2}[0-9]{4}', reg). It is concise and O(L) for this fixed pattern, while slicing makes each field explicit. Add the actual six-case output screenshot.

Q4 – Building Long Text: + or join()? (6 Marks)

Bloom's Level: K4 – Analyse

Strings are immutable, so repeated concatenation may copy intermediate strings. join() is generally preferred for joining many fragments.

4.1 Implementations and Benchmark

```
import time

def build_plus(parts):
    text = ""
    for p in parts:
        text += p + ";"
    return text

def build_join(parts):
    return ";".join(parts) + (";" if parts else "")

def build_fstring(parts):
    text = ""
    for part in parts:
        text += f"{part};"
    return text

def measure(func, items, repeats=3):
    times = []
    result = ""
    for _ in range(repeats):
        start = time.perf_counter()
        result = func(items)
        times.append(time.perf_counter() - start)
    return result, sum(times) / len(times), times

for n in (1000, 10000, 200000):
    items = [f"Item{i}:Rs{i % 50}" for i in range(n)]
```

```
    a, avg_plus, raw_plus = measure(build_plus, items)
    b, avg_join, raw_join = measure(build_join, items)
    c, avg_f, raw_f = measure(build_fstring, items)
    print(n, a == b == c, len(a))
    print("+= average/runs:", avg_plus, raw_plus)
    print("join average/runs:", avg_join, raw_join)
    print("f-string average/runs:", avg_f, raw_f)
```

Lines	+= average (s)	join average (s)	f-string average (s)	Same text?
1,000	[record]	[record]	[record]	[Yes/No]
10,000	[record]	[record]	[record]	[Yes/No]
200,000	[record]	[record]	[record]	[Yes/No]

Run three trials per size and use the printed averages. Do not invent timings: they depend on the computer and system load. join() is O(total output length) and usually avoids repeated intermediate copies; repeated += can approach $O(n^2)$ in unfavourable cases, although CPython sometimes optimises it. Writing to a file or using io.StringIO can reduce the need to hold the complete final string in memory. Attach your measured table and screenshot.

Q5 – Functions, Parameters and Scope (7 Marks)

Bloom's Level: K4 – Analyse

5.1 Mutable Default Argument

The order=[] default list is created once when the function is defined, so calls share the same list. Use None and create a new list inside the function.

```
def add_item_buggy(item, order=[]):
    order.append(item)
    return order

def add_item(item, order=None):
    if order is None:
        order = []
    order.append(item)
    return order

print(add_item_buggy("Tea"), add_item_buggy("Vada"))
print(add_item("Tea"), add_item("Vada"))
print(add_item_buggy.__defaults__)
```

Expected output:

```
['Tea', 'Vada'] ['Tea', 'Vada']
['Tea'] ['Vada']
(['Tea', 'Vada'],)
```

5.2 Parameters, Return Values and Scope

- Positional parameters suit simple calls such as add_item('Tea').
- Keyword parameters improve readability, such as calculate_total(price=100, quantity=2).
- Defaults are useful for optional arguments, but mutable objects should not be used as defaults.
- Multiple values can be returned as a tuple: return total, gst.

- Local names belong to a function; global names belong to the module; nonlocal refers to a binding in an enclosing function.

```
GST_RATE = 0.05

def calculate_total(price, quantity=1):
    subtotal = price * quantity
    gst = subtotal * GST_RATE
    return subtotal, gst

def scope_example():
    count = 1
    def inner():
        nonlocal count
        count += 1
    inner()
    return count

print(calculate_total(100, quantity=2))
print(scope_example())
```

Changing a global name requires the global keyword, but global state is usually avoided because it makes functions harder to test. Passing an order explicitly is another good design. Appending one item is O(1) amortised; storing m items for a customer uses O(m) space. Test three customers and show their lists remain independent.

Q6 – Recursion and Its Limits (8 Marks)

Bloom's Level: K5 – Evaluate

For n steps, where each move is one or two steps, W(n)=W(n−1)+W(n−2), with W(0)=W(1)=1.

6.1 Recursive and Memoised Versions

```
from functools import lru_cache
import sys
from time import perf_counter

calls = 0
def ways_naive(n):
    global calls
    calls += 1
    if n <= 1:
        return 1
    return ways_naive(n - 1) + ways_naive(n - 2)

calls_memo = 0
@lru_cache(maxsize=None)
def ways_memo(n):
    global calls_memo
    calls_memo += 1
    if n <= 1:
        return 1
    return ways_memo(n - 1) + ways_memo(n - 2)

for n in range(1, 7):
    ways_memo.cache_clear()
    print(n, ways_memo(n))

for n in (10, 20, 25, 30):
    calls = 0
```

```
    start = perf_counter()
    naive_answer = ways_naive(n)
    naive_time = perf_counter() - start
    ways_memo.cache_clear()
    calls_memo = 0
    start = perf_counter()
    memo_answer = ways_memo(n)
    memo_time = perf_counter() - start
    print(n, naive_answer, calls, naive_time, memo_answer, calls_memo, memo_time)

print("Recursion limit:", sys.getrecursionlimit())
ways_memo.cache_clear()
try:
    print(ways_memo(1000))
except RecursionError as error:
    print("RecursionError:", error)
```

6.2 Call Tree for n=5

```
ways(5)
├── ways(4)
│   ├── ways(3)
│   │   ├── ways(2)
│   │   └── ways(1)
│   └── ways(2)
└── ways(3)
    ├── ways(2)
    └── ways(1)
```

n	W(n)
1	1
2	2
3	3
4	5
5	8
6	13

6.3 Iterative Alternative

```
def ways_loop(n):
    if n <= 1:
        return 1
    previous, current = 1, 1
    for _ in range(2, n + 1):
        previous, current = current, previous + current
    return current

print("ways_loop(1000):", ways_loop(1000))
```

Plain recursion takes $O(\phi^n)$ time and O(n) stack space. Memoisation takes O(n) time with O(n) cache and stack space, but a recursive call depth near 1,000 may exceed sys.getrecursionlimit(). The iterative solution takes O(n) time and O(1) auxiliary space, so it is the recommended choice for n=1,000. Record actual calls, timings, recursion limit and iterative output.

Q7 – Diagnose, Redesign and Validate (13 Marks)

Bloom's Level: K5 – Evaluate

Selected program: Q6, counting ways to climb temple steps. Plain recursion repeats the same subproblems and grows exponentially.

A. Diagnose the Bottleneck (2 Marks)

The calls ways_naive(n-1) and ways_naive(n-2) repeatedly calculate identical values. The assignment's reference output for n=25 is 121,393 ways and 242,785 calls; confirm these values by running the program.

B. Design Objective (2 Marks)

Return the same correct result while reducing time from exponential to linear and complete n=1,000 without a recursion-depth failure.

C. Revised Solution (4 Marks)

```
def ways_redesigned(n):
    if n <= 1:
        return 1
    previous, current = 1, 1
    for _ in range(2, n + 1):
        previous, current = current, previous + current
    return current

for n in (10, 20, 25, 30, 1000):
    print(n, ways_redesigned(n))
```

Pseudocode: initialise previous=1 and current=1; for each step from 2 through n, calculate next=previous+current and shift the two values; return current. The algorithm needs O(n) additions and O(1) auxiliary variables, excluding the growing integer result.

D. Prediction Before Implementation (1 Mark)

The iterative solution should be substantially faster than plain recursion for n≥25 and should complete n=1,000 without RecursionError.

E. Implement and Validate (2 Marks)

n	Naive calls/time	Redesigned time	Same answer?	Observation
10	[record]	[record]	[Yes/No]	[record]
20	[record]	[record]	[Yes/No]	[record]
25	[record]	[record]	[Yes/No]	[record]
30	[record]	[record]	[Yes/No]	[record]
1000	Not recommended	[record]	Compare memo/loop	Loop should complete

Measure with time.perf_counter() on the same computer. Avoid running naive recursion at n=1,000. Replace placeholders with actual results.

F. Defend the Final Design (2 Marks)

The redesign succeeds if it returns matching answers for the tested inputs and completes n=1,000. It is less directly expressive of the recurrence than recursion, but avoids repeated work and stack-depth failure. Plain recursion remains useful for illustrating the mathematical definition at small n.

Conclusion

This assignment applies decisions, loops, string normalisation and validation, efficient string building, functions, scope, recursion and complexity analysis. Clear boundary conditions, independent data structures and avoiding repeated computation improve correctness and scalability.

Submission Checklist

- Create repository CSA08-1912260031-Assignment2.
- Keep each question's source code, inputs, outputs and screenshots in a separate folder.
- Include README.md with student details, Python version, libraries and run commands.
- Commit progressively, at least once per question.
- Replace measurement placeholders with actual counts, timings and observations.
- Add execution screenshots and verify every program.
- Paste the working Assignment 2 GitHub link on the cover page.